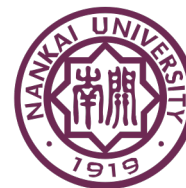


第5讲 OpenMP共享内存编程



南开大学软件学院

NANKAI UNIVERSITY
COLLEGE OF SOFTWARE



南开大学
Nankai University

■ 第五章 OpenMP共享内存编程

- 5.1 预备知识
- 5.2 梯形积分法
- 5.3 变量的作用域
- 5.4 归约子句
- 5.5 parallel for
- 5.6 更多OpenMP的循环：排序
- 5.7 循环调度
- 5.8 生产者和消费者问题

目录

1. OpenMP并行模型
2. 并行循环
3. 数据依赖
4. 循环调度
5. 局部性
6. 任务并行

目录

1. OpenMP并行模型

2. 并行循环

3. 数据依赖

4. 循环调度

5. 局部性

6. 任务并行

1.1 OpenMP编程模型概述

- 是Pthread的常见替代，更简单，但限制也更多
 - 通过少量编译指示指出并行部分和数据共享，即可实现很多串行程序的并行化
- 可移植：不同共享内存架构
- 可扩展
- 增量并行化：程序不同部分逐步并行化
- 依赖编译器生成线程创建和管理代码
 - 语言扩展：C、C++、Fortran
主要是编译指示、少量库函数

官网 <http://www.openmp.org>

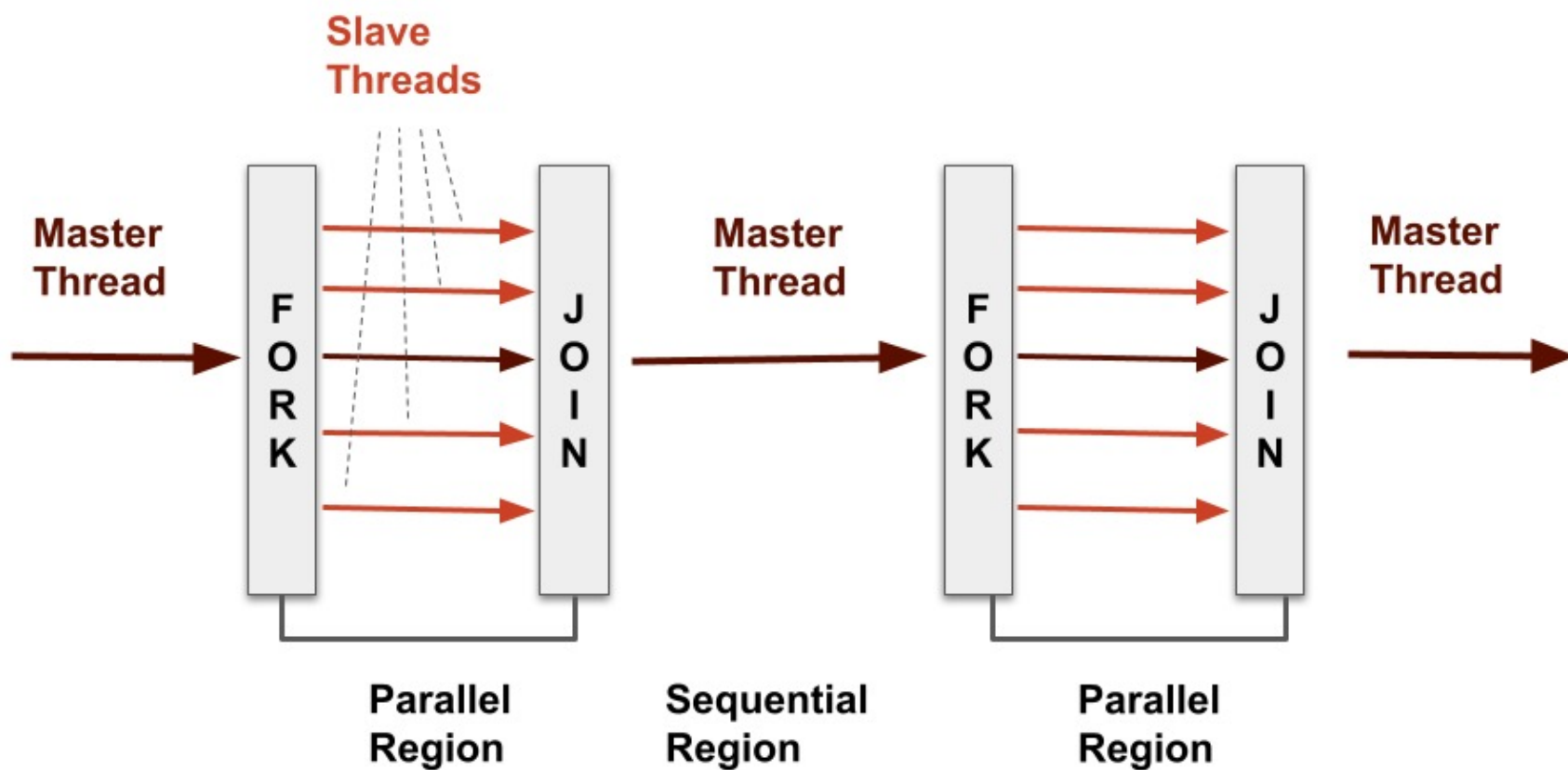
1.2 OpenMP：程序员视角

- 一种可移植共享内存多线程编程规范，“轻量”语法
 - 准确行为依赖于具体实现
 - 需要编译器支持（C、C++、Fortran）
- OpenMP能
 - 程序员只需将程序分为串行和并行区域，而不是构建并发执行的多线程
 - 隐藏栈管理
 - 提供同步机制
- OpenMP不能
 - 自动并行化
 - 确保加速
 - 避免数据竞争

1.3 OpenMP执行模型

- Fork-join并行执行模型
- 执行伊始是单线程（**主线程**）
- 并行结构开始
 - 主线程创建一组线程（**工作线程**）
- 并行结构结束
 - 线程组同步——**隐式barrier**
- 只有主线程继续执行
- 实现优化
 - 工作线程等待下一次fork

1.3 OpenMP执行模型



1.4 使用编译指示

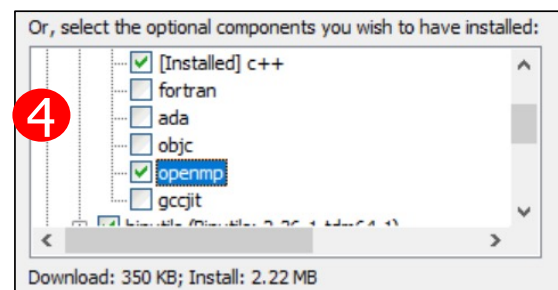
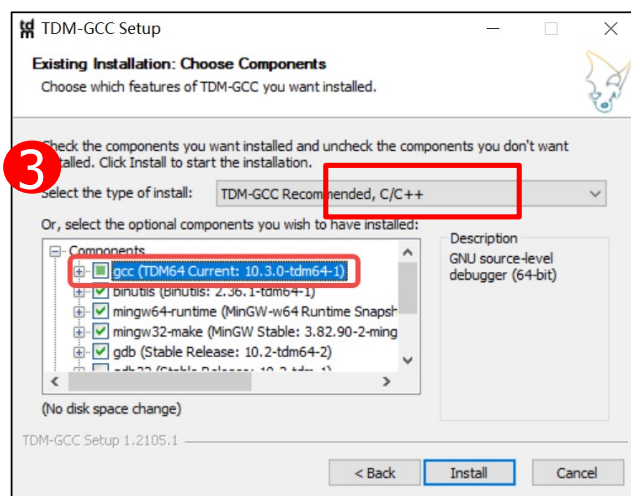
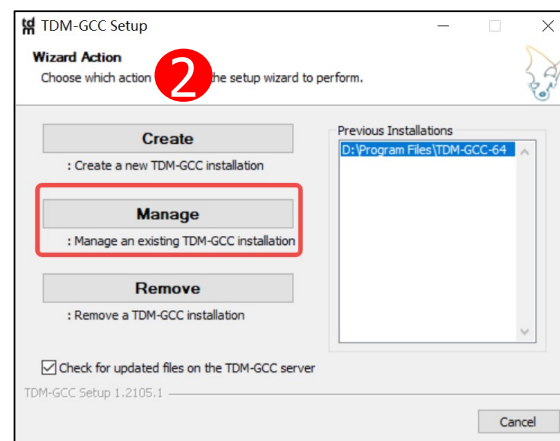
- 编译指示 (pragma) 是一些特殊的预处理指令
- 为了提供标准C规范之外的功能
- 编译器会忽略不支持的编译指示
- OpenMP编译指示的解释
 - 修改紧跟其后的语句——可能是循环这样的复合语句

`#pragma omp ...`

1.5 Code::Blocks+TDM-GCC环境配置

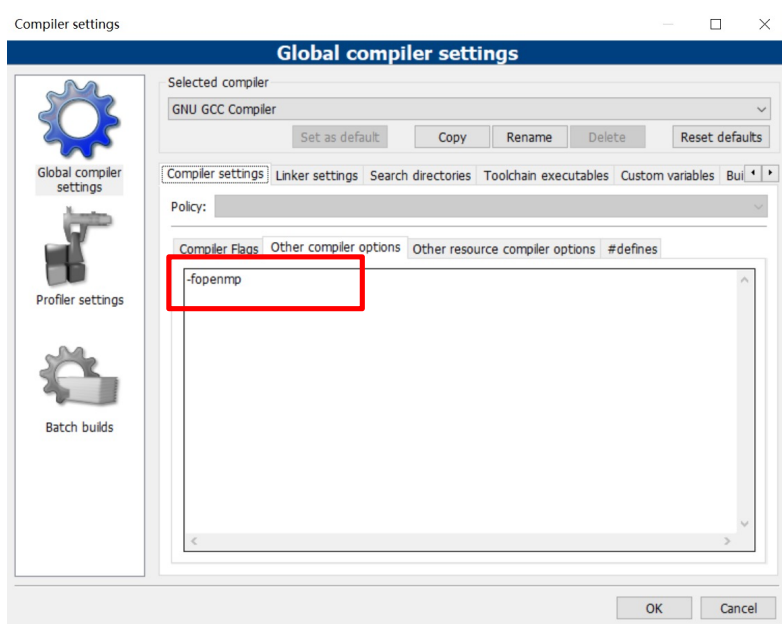
■ TDM-GCC安装需要勾选openmp

如果已经安装，按下面步骤执行

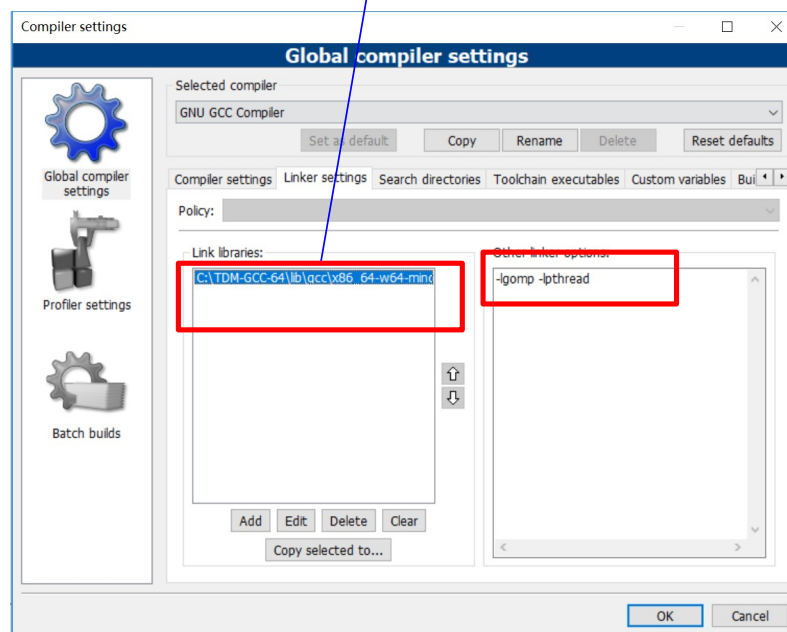


1.5 Code::Blocks+TDM-GCC环境配置

Code ::Blocks编译器设置



C:\TDM-GCC-64\lib\gcc\x86_64-w64-mingw32\5.1.0\libgomp.dll.a



1.6 编程模型 – 数据共享

■ 并行程序通常使用两种数据

- 共享数据，所有线程可见，通常是具名的
- 私有数据，单线程可见，通常在栈中分配

■ PThread

- 全局作用域变量是共享的
- 栈中分配的变量是私有的

■ OpenMP

- shared变量是共享的
- private变量是私有的
- 默认是shared
- 循环变量是private

```
// 共享全局变量  
int bigdata[1024];
```

```
void* foo(void* bar) {  
    int tid;
```

```
#pragma omp parallel \  
shared ( bigdata ) \  
private ( tid )  
{  
    /* Calc. here */  
}
```

1.7 编译指示格式

■ 编译指示格式

```
#pragma omp directive_name [ clause [ clause ] ... ]
```

■ 指令 (directive_name) 形式

- 并行区 : parallel
- 分工 : for, sections, single
- 同步 : barrier, critical, atomic

```
#pragma omp parallel for num_threads(4)  
#pragma omp for nowait  
#pragma omp atomic
```

■ 子句 (clause) 形式

- num_threads(4)
- shared(list)
- private(list)
- reduction(operator:list)
- schedule(type[, chunk])
- nowait (C/C++ : 用于#pragma omp for)

1.7 编译指示格式

■ 条件编译

```
#ifdef _OPENMP
```

```
...
```

```
    printf(“%d avail.processors\n”,omp_get_num_procs());
```

```
#endif
```

■ 大小写敏感

■ 使用库函数需要包含头文件

```
#ifdef _OPENMP
```

```
#include <omp.h>
```

```
#endif
```

1.8 查询函数

- 返回执行当前并行区域的线程组中的线程数

```
int omp_get_num_threads(void);
```

- 返回当前线程在线程组中的编号，值在0和
omp_get_num_threads()-1之间。主线程的编号为0

```
int omp_get_thread_num(void);
```

1.9 并行区域结构

- 多线程并行执行的代码块
- 每个线程执行相同的代码 (**SPMD**)
 - 线程组中线程分担工作，任务被分配给它们

- C/C++ 语法示例

```
#pragma omp directive_name [ clause [ clause ] ... ]  
语句块
```

- 子句可为下面情况
 - `private (list)`
 - `shared (list)`

1.10 Hello World

- 我们从一个简单的并行区域结构开始
- 需要考虑的问题
 - 从命令行读取线程数
 - 没有pragma和库调用的代码应该是正确的
- 与Pthread代码的区别
 - 更多必要的代码是由编译器和运行时环境管理的
 - 有隐含的线程标识

`gcc -fopenmp ...`

1.10 Hello World

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
void Hello(void); /* Thread function */

int main(int argc, char* argv[]) {
    /* Get number of threads from command line */
    int thread_count = strtol(argv[1], NULL, 10);

    # pragma omp parallel num_threads(thread_count)
    Hello();

    return 0;
} /* main */
```

1.10 Hello World

```
void Hello(void) {  
    int my_rank = omp_get_thread_num();  
    int thread_count = omp_get_num_threads();  
  
    printf("Hello from thread %d of %d\n", my_rank, thread_count);  
  
} /* Hello */
```

1.10 Hello World

```
Hello from thread 2 of 4  
Hello from thread 0 of 4  
Hello from thread 1 of 4  
Hello from thread 3 of 4
```

1.11 为防止编译器不支持OpenMP

```
# include <omp.h>
```

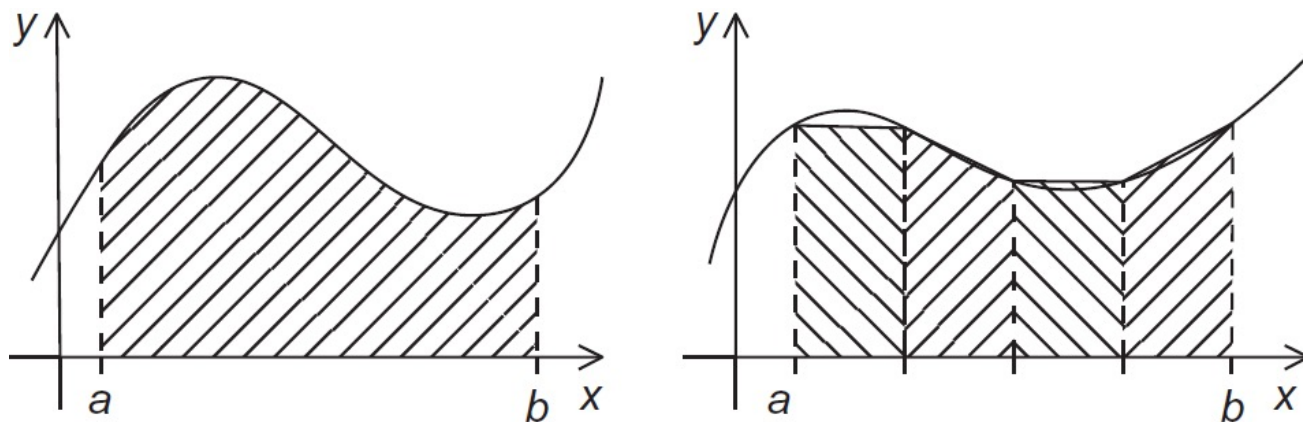


```
#ifdef _OPENMP  
# include <omp.h>  
#endif
```

1.11 为防止编译器不支持OpenMP

```
# ifdef _OPENMP
    int my_rank = omp_get_thread_num ( );
    int thread_count = omp_get_num_threads ( );
# else
    int my_rank = 0;
    int thread_count = 1;
# endif
```

1.12 梯形积分法：函数 $f(x)$ $[a,b]$ 间积分



■ 间隔 $h = x_{i+1} - x_i = (b-a)/n$

■ 每个梯形面积 $\frac{h}{2}[f(x_i) + f(x_{i+1})]$

$$x_0 = a, x_1 = a + h, x_2 = a + 2h, \dots, x_{n-1} = a + (n-1)h, x_n = b$$

■ 梯形面积之和

$$: h[f(x_0)/2 + f(x_1) + f(x_2) + \dots + f(x_{n-1}) + f(x_n)/2]$$

1.12 梯形积分法：串行版本

```
/* Input: a, b, n */
```

```
h = (b - a)/n;
```

```
approx = (f(a) + f(b))/2.0;
```

```
for (i = 1; i <= n-1; i++)
```

```
    approx += f(a + i*h);
```

```
approx = h * approx;
```

多线程求全局和
怎么保证正确性？



1.13 临界区指令

■ 被临界区包围的代码

- 所有线程都执行，但每个时刻限制只有一个线程执行

```
#pragma omp critical [ ( name ) ]
```

语句块

- 线程在临界区开始位置等待，直至组中没有其他线程在同名临界区中执行
- 所有未命名临界区指示都映射到相同的未指定名字

1.12 梯形积分法：第一个OpenMP版本

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
void Trap( double a, double b, int n, double* global_result_p);
int main(int argc, char* argv[]) {
    double global_result = 0.0;
    double a, b;
    int n;
    int thread_count ;

    thread_count = strtol(argv[1], NULL, 10) ;
    printf("Enter a, b, and n\n");
    scanf("%lf %lf %d", &a, &b, &n);

    # pragma omp parallel num_threads(thread_count)
    Trap(a, b, n, &global_result);
    printf("With n = %d trapezoids, our estimate\n", n);
    printf("of the integral from %f to %f = %.14e\n", a, b,
global_result);
    return 0;
} /* main*/
```

1.12 梯形积分法：第一个OpenMP版本(2)

```

void Trap( double a, double b, int n, double* global_result_p )
{
    double h, x, my_result ;
    double local_a , local_b ;
    int i, local_n ;
    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();
    h = (b-a)/n;
    local_n = n/thread_count ;
    local_a = a + my_rank*local_n*h;
    local_b = local_a + local_n*h;
    my_result = (f(local_a) + f(local_b))/2.0;
    for (i = 1; i <= local_n - 1; i++) {
        x = local_a + i*h;
        my_result += f(x);
    }
    my_result = my_result*h;
    # pragma omp critical
    *global_result_p += my_result ;
} /* Trap*/

```

先局部求和
避免过多通信

用临界区保证
全局求和正确性

1.12 梯形积分法：改得更简洁

```
global_result = 0.0;
...
# pragma omp parallel num_threads(thread_count)
  Trap(a, b, n, &global_result);
...
```

有什么问题？



```
global_result = 0.0;
# pragma omp parallel num_threads(thread_count)
{
#   pragma omp critical
    global_result += Local_trap(a, b, n);
}
```

不同线程的全部计算
被串行化了

1.12 梯形积分法：避免所有计算串行化

```
global_result = 0.0;
# pragma omp parallel num_threads(thread_count)
{
#   pragma omp critical
    global_result += Local_trap(a, b, n);
}
```



```
global_result = 0.0;
# pragma omp parallel num_threads(thread_count)
{
    double my_result = 0.0; /* private */
    my_result += Local_trap(a, b, n);
# pragma omp critical
    global_result += my_result;
}
```

不同线程并行计算
只有全局求和串行

1.14 OpenMP归约

■ OpenMP支持归约操作

- 归约就是将相同的规约操作符重复地应用到操作数序列来得到一个结果的计算。

```
sum = 0;
#pragma omp parallel for reduction(+:sum)
for (i=0; i < 100; i++) {
    sum += array[i];
}
```

■ 支持的归约运算及初值

+	0	位与 &	~0	逻辑与 &	1
-	0	位或	0	逻辑或	0
*	1	位异或 ^	0		

1.12 梯形积分法：改为使用归约

```
global_result = 0.0;
# pragma omp parallel num_threads(thread_count)
{
    double my_result = 0.0; /* private */
    my_result += Local_trap(a, b, n);
# pragma omp critical
    global_result += my_result;
}
```



**私有变量等繁琐工作编译器代劳
全局求和采用递归算法**

```
global_result = 0.0;
# pragma omp parallel num_threads(thread_count) \
    reduction(+: global_result)
    global_result += Local_trap(a, b, n);
```

目录

1. OpenMP并行模型

2. 并行循环

3. 数据依赖

4. 循环调度

5. 局部性

6. 任务并行

2.1 OpenMP数据并行：并行循环

- 所有编译指示都以#pragma开始
- 编译器为每个线程计算负责的循环范围——根据串行源码（计算分解）
- 编译器还管理数据划分
- 同步也是自动的（barrier）

Serial Program:

```
void main()
{
    double Res[1000];

    for(int i=0;i<1000;i++) {
        do_huge_comp(Res[i]);
    }
}
```

Parallel Program:

```
void main()
{
    double Res[1000];
    #pragma omp parallel for
    for(int i=0;i<1000;i++) {
        do_huge_comp(Res[i]);
    }
}
```

2.2 局限和语义

■ 并非所有“元素级”循环都能并行化

```
#pragma omp parallel for
for (i=0; i < numPixels; i++) {}
```

- 循环变量：带符号整数
- 终止检测：<, <=, >, >=与循环不变量
- 每步迭代递增/递减一个循环不变量
- 对<, <=向上计数；对>, >=向下计数
- 循环体：无进/出控制流

■ 创建线程，在线程间分配循环步；要求迭代次数可预测。

- 不检查依赖性！
- 不支持while、do-while、循环体包含break等

for	index = start ;	;	index < end	;	index >= end ;	;	index++
							++index
							index--
							--index
							index += incr
							index -= incr
							index = index + incr
							index = incr + index
							index = index - incr

2.2 局限和语义

■ 要求迭代次数可预测

```
for ( ; ; ) {  
    . . .  
}
```

```
for (i = 0; i < n; i++) {  
    if ( . . . ) break;  
    . . .  
}
```

不允许！

■ 无限循环

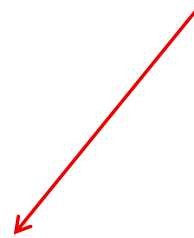
■ 循环中途退出

2.3 简单的并行化循环

```
/* Input: a, b, n */
h = (b - a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++)
    approx += f(a + i*h);
approx = h * approx;
```



```
global_result = 0.0;
# pragma omp parallel num_threads(thread_count)
    reduction(+: global_result)
    global_result += Local_trap(a, b, n);
```



```
/* Input: a, b, n */
h = (b - a)/n;
approx = (f(a) + f(b))/2.0;
#pragma omp parallel for num_threads(thread_count) \
    reduction(+: approx)
for (i = 1; i <= n-1; i++)
    approx += f(a + i*h);
approx = h * approx;
```

2.4 同步

■ 隐式barrier

- 并行结构开始和结束位置
- 其他控制结构的结束位置
- 用nowait子句可去除隐式同步

2.4 同步

隐式barrier机制

```
#pragma omp parallel num_threads(4)
{
    int tid = omp_get_thread_num();

    #pragma omp for
    for (int i = 0; i < 4; i++) {
        printf("[No nowait] Thread %d processing item %d\n", tid, i);
        if (i == 0) {
            sleep(3); // 第一个任务故意慢3秒
            printf("[No nowait] Thread %d: item 0 done (slow)\n", tid);
        }
    }
    // 隐式Barrier

    printf("[No nowait] Thread %d continues after barrier\n", tid);
}
```

2.4 同步

隐式barrier机制

```
#pragma omp parallel num_threads(4)
{
    int tid = omp_get_thread_num();

    #pragma omp for
    for (int i = 0; i < 4; i++) {
        printf("[No nowait] Thread %d processing item %d\n", tid, i);
        if (i == 0) {
            sleep(3); // 第一个任务故意慢3秒
            printf("[No nowait] Thread %d: item 0 done (slow)\n", tid);
        }
    }
    // 隐式Barrier

    printf("[No nowait] Thread %d continues after barrier\n", tid);
}
```

[No nowait] Thread 0 processing item 0
[No nowait] Thread 3 processing item 3
[No nowait] Thread 2 processing item 2
[No nowait] Thread 1 processing item 1
[No nowait] Thread 0: item 0 done (slow)
[No nowait] Thread 0 continues after barrier
[No nowait] Thread 2 continues after barrier
[No nowait] Thread 3 continues after barrier
[No nowait] Thread 1 continues after barrier

2.4 同步

nowait子句去除隐式同步

```
#pragma omp parallel num_threads(4)
{
    int tid = omp_get_thread_num();

    #pragma omp for nowait
    for (int i = 0; i < 4; i++) {
        printf("[With nowait] Thread %d processing item %d\n", tid, i);
        if (i == 0) {
            sleep(3); // 第一个任务故意慢3秒
            printf("[With nowait] Thread %d: item 0 done (slow)\n", tid);
        }
    }
    // 没有barrier！完成的线程直接继续

    printf("[With nowait] Thread %d continues immediately\n", tid);
}
```


2.4 同步

nowait子句去除隐式同步

```
#pragma omp parallel num_threads(4)
{
    int tid = omp_get_thread_num();

    #pragma omp for nowait
    for (int i = 0; i < 4; i++) {
        printf("[With nowait] Thread %d processing item %d\n",
            tid, i);
        if (i == 0) {
            sleep(3); // 第一个任务故意慢3秒
            printf("[With nowait] Thread %d: item 0 done
(slow)\n", tid);
        }
        // 没有barrier! 完成的线程直接继续

        printf("[With nowait] Thread %d continues immediately\n",
            tid);
    }
}
```

[With nowait] Thread 0 processing item 0
 [With nowait] Thread 1 processing item 1
 [With nowait] Thread 2 processing item 2
 [With nowait] Thread 2 continues immediately
 [With nowait] Thread 1 continues immediately
 [With nowait] Thread 3 processing item 3
 [With nowait] Thread 3 continues immediately
[With nowait] Thread 0: item 0 done (slow)
 [With nowait] Thread 0 continues immediately

2.4 同步

■ 显式同步

- critical
- atomic (单一语句)
- barrier

2.4 同步

临界区 critical

```
int sum = 0;

#pragma omp parallel num_threads(4)
{
    int thread_id = omp_get_thread_num();

    #pragma omp critical
    {
        sum += thread_id;
        printf("Thread %d is updating sum  
to %d\n", thread_id, sum);
    }
}

printf("Final sum = %d\n", sum);
```

不加临界区

Thread 0 is updating sum to 5
Thread 3 is updating sum to 5
Thread 2 is updating sum to 2
Thread 1 is updating sum to 3
Final sum = 5



加临界区

Thread 0 is updating sum to 0
Thread 3 is updating sum to 3
Thread 1 is updating sum to 4
Thread 2 is updating sum to 6
Final sum = 6

2.4 同步

原子操作 atomic

```
int counter = 0;

#pragma omp parallel for num_threads(4)
for (int i = 0; i < 1000; i++) {
    #pragma omp atomic
    counter++;
}

printf("Counter = %d\n", counter);
```

无原子操作

Counter = 381

**原子操作**

Counter = 1000

2.4 同步

如果用critical代替atomic?

```
int counter = 0;

#pragma omp parallel for num_threads(4)
for (int i = 0; i < 10000000; i++) {
    #pragma omp atomic/critical
    counter++;
}

printf("Counter = %d\n", counter);
```

■ 使用 critical

结果: 10000000
时间: 1.395 秒

■ 使用 atomic

结果: 10000000
时间: 0.228 秒

■ 不加保护 (会出错)

结果: 2694652 (错误)
时间: 0.035 秒

■ 性能对比:

atomic 比 critical 快 6.1 倍

2.4 同步

屏障 barrier

```
#pragma omp parallel num_threads(4)
{
    int thread_id =
omp_get_thread_num();

    // 第一阶段
    sleep(thread_id); // 模拟不同工作时长
    printf("Thread %d: Phase 1 done\n",
thread_id);

    // 等待所有线程完成第一阶段
    #pragma omp barrier

    // 第二阶段工作 ( 所有线程同时开始 )
    printf("Thread %d: Phase 2 start\n",
thread_id);
}
```

不加barrier

Thread 0: Phase 1 done
Thread 0: Phase 2 start
Thread 1: Phase 1 done
Thread 1: Phase 2 start
Thread 2: Phase 1 done
Thread 2: Phase 2 start
Thread 3: Phase 1 done
Thread 3: Phase 2 start



加barrier

Thread 0: Phase 1 done
Thread 1: Phase 1 done
Thread 2: Phase 1 done
Thread 3: Phase 1 done
Thread 3: Phase 2 start
Thread 0: Phase 2 start
Thread 1: Phase 2 start
Thread 2: Phase 2 start

2.5 并行for指示的各种形式

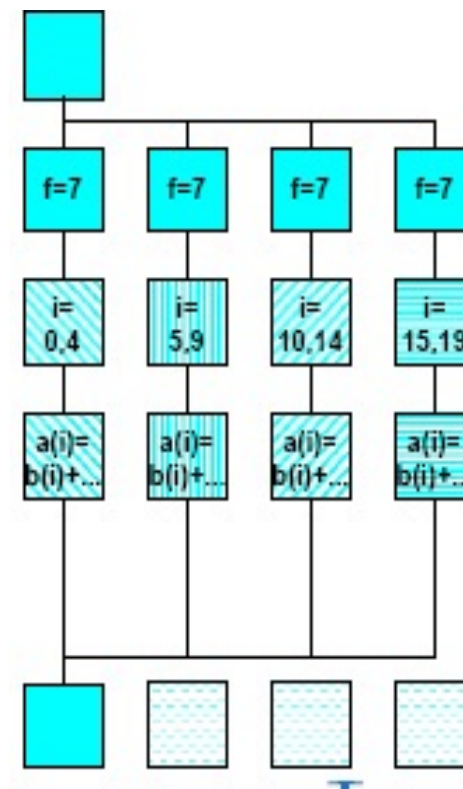
■ 语法

```
#pragma omp for [ clause [ clause ] ... ]  
for循环
```

■ 子句形式

- shared(list)
- private(list)
- reduction(operator:list)
- schedule(type[, chunk])
- nowait (C/C++ : 用于#pragma omp for)

```
#pragma omp parallel private(f) {  
    f=7;  
    #pragma omp for  
    for (i=0; i<20; i++)  
        a[i] = b[i] + f * (i+1);  
} /* omp end parallel */
```



目录

1. OpenMP并行模型
2. 并行循环
- 3. 数据依赖**
4. 循环调度
5. 局部性
6. 任务并行

3.1 竞争条件与数据依赖

- 执行结果依赖于两个或更多事件的时序，则存在**竞争条件**（race condition）
- **数据依赖**（data dependence）就是两个内存操作的序，为了保证结果的正确性，必须保持这个序
 - 如果两个内存访问指向相同的内存位置且其中一个是写操作，则它们产生数据依赖

3.2 一些定义、定理 (Allen & Kennedy)

■ 一些定义

- 两个计算等价 $\leftarrow$ 在相同的输入上
 - 它们产生相同的输出
 - 输出按相同的顺序生成
- 一个重排转换
 - 改变语句执行的顺序
 - 不增加或删除任何语句的执行
- 一个重排转换保持依赖关系
 - 它保持了依赖源和目的语句的相对执行顺序

■ 依赖关系基本定理

- 任何重排转换，只要保持了程序中所有依赖关系，它就保持了程序的含义

3.3 重排转换

■ 并行化

- 同步点之间并行执行的计算可能重排执行顺序。这种重排是否安全？根据我们的定义，如果它能保持代码中的依赖关系，则它是安全的

■ 局部性优化

- 假定我们希望修改访问顺序，以便更好地利用cache。这也是一种重排转换，同样，若它保持了代码中的依赖关系，则它是安全的

■ 归约计算

- 对于使用满足交换律和结合律的运算的归约操作，对其重排是安全的

3.4 数组的数据依赖

```
for (i=2; i<5; i++)  
    A[i] = A[i-2]+1;
```

循环进位依赖关系

```
for (i=1; i<=3; i++)  
    A[i] = i+1;  
    B[i] = A[i]*3
```

循环独立关系

■ 识别并行循环（直观地）

- 寻找循环中的数据依赖
- 没有依赖关系跨越迭代步边界 → 并行化是安全的

3.5 估算 π

$$\pi = 4 \left[1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots \right] = 4 \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}$$

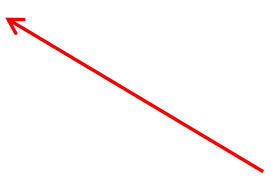
```
double factor = 1.0;
double sum = 0.0;
for (k = 0; k < n; k++) {
    sum += factor/(2*k+1);
    factor = -factor;
}
pi_approx = 4.0*sum;
```

3.5 估算 π ：第一个OpenMP版本

```
double factor = 1.0;
double sum = 0.0;
# pragma omp parallel for num_threads(thread_count) \
    reduction(+:sum)
for (k = 0; k < n; k++) {
    sum += factor/(2*k+1);
    factor = -factor;
}
pi_approx = 4.0*sum;
```

有什么问题？

一条语句由多个线程
并行执行产生数据依赖



3.5 估算 π ：根据k的奇偶性计算factor

- k为奇数，factor为-1；k为偶数，factor为+1

```
double factor = 1.0;
double sum = 0.0;
# pragma omp parallel for
num_threads(thread_count) \
    reduction(+:sum)
for (k = 0; k < n; k++) {
    factor = (k % 2 == 0) ? 1.0 : -1.0;
    sum += factor/(2*k+1);
}
pi_approx = 4.0*sum;
```

仍是错误的！

factor应是私有的！

3.6 估算 π : factor变为私有

- k为奇数，factor为-1；k为偶数，factor为+1

```
double factor = 1.0;
double sum = 0.0;
# pragma omp parallel for
num_threads(thread_count) \
    reduction(+:sum) private(factor)
for (k = 0; k < n; k++) {
    factor = (k % 2 == 0) ? 1.0 : -1.0;
    sum += factor/(2*k+1);
}
pi_approx = 4.0*sum;
```


3.7 冒泡排序

```
for (list_length= n; list_length >= 2;
list_length--)
    for (i = 0; i < list_length-1; i++)
        if (a[i] > a[i+1]) {
            tmp = a[i];
            a[i] = a[i+1];
            a[i+1] = tmp;
        }
```

外层循环：依赖关系跨越迭代步边界

内层循环迭代步之间也有依赖

能否直接并行化？

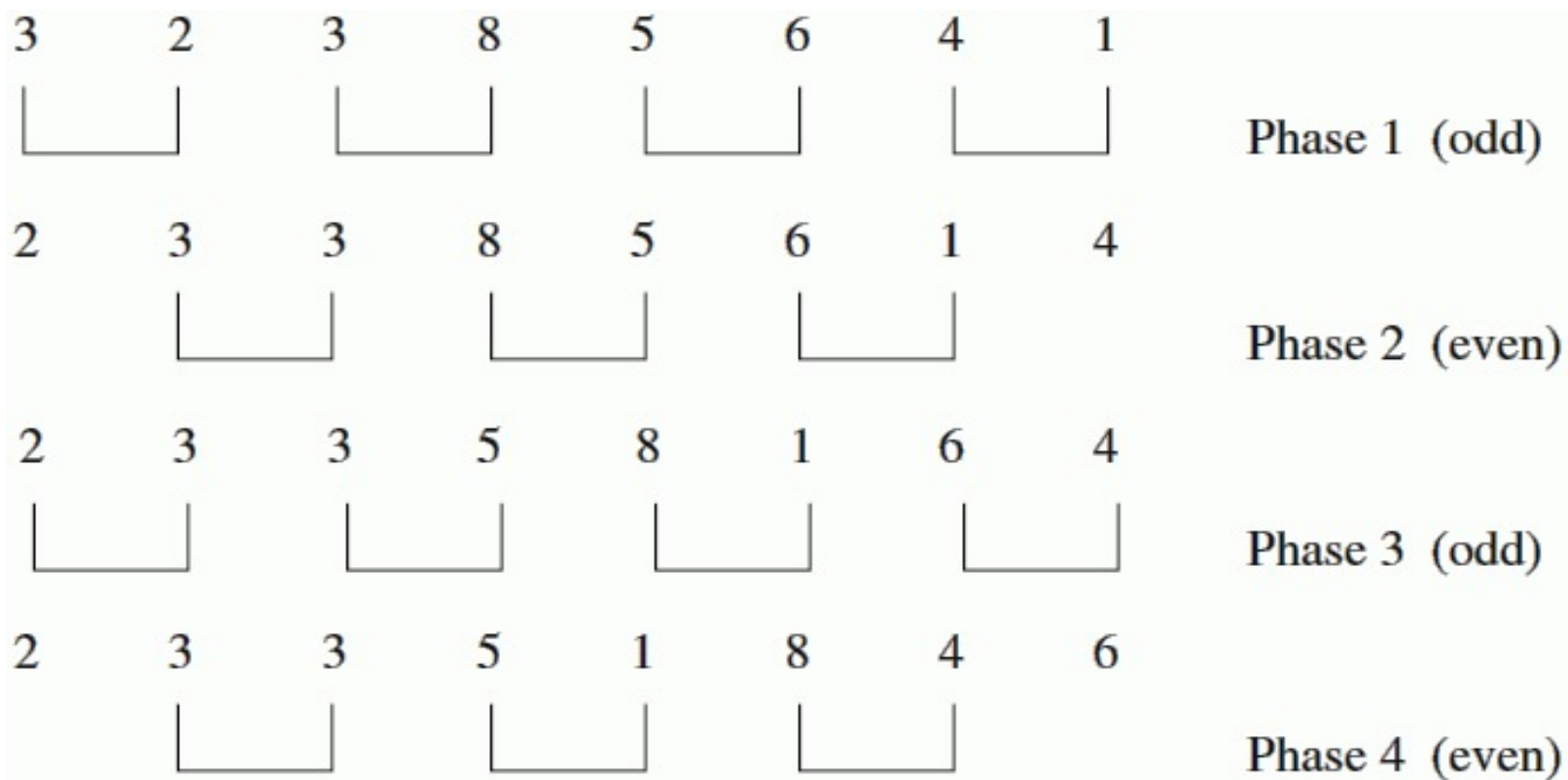
■ 例：a=[3,4,1,2]

- 外循环，每次循环“下沉”最大数。第一步后 [3,1,2,4]
- 内循环：每次用到a[i]和a[i+1]，明显存在循环依赖

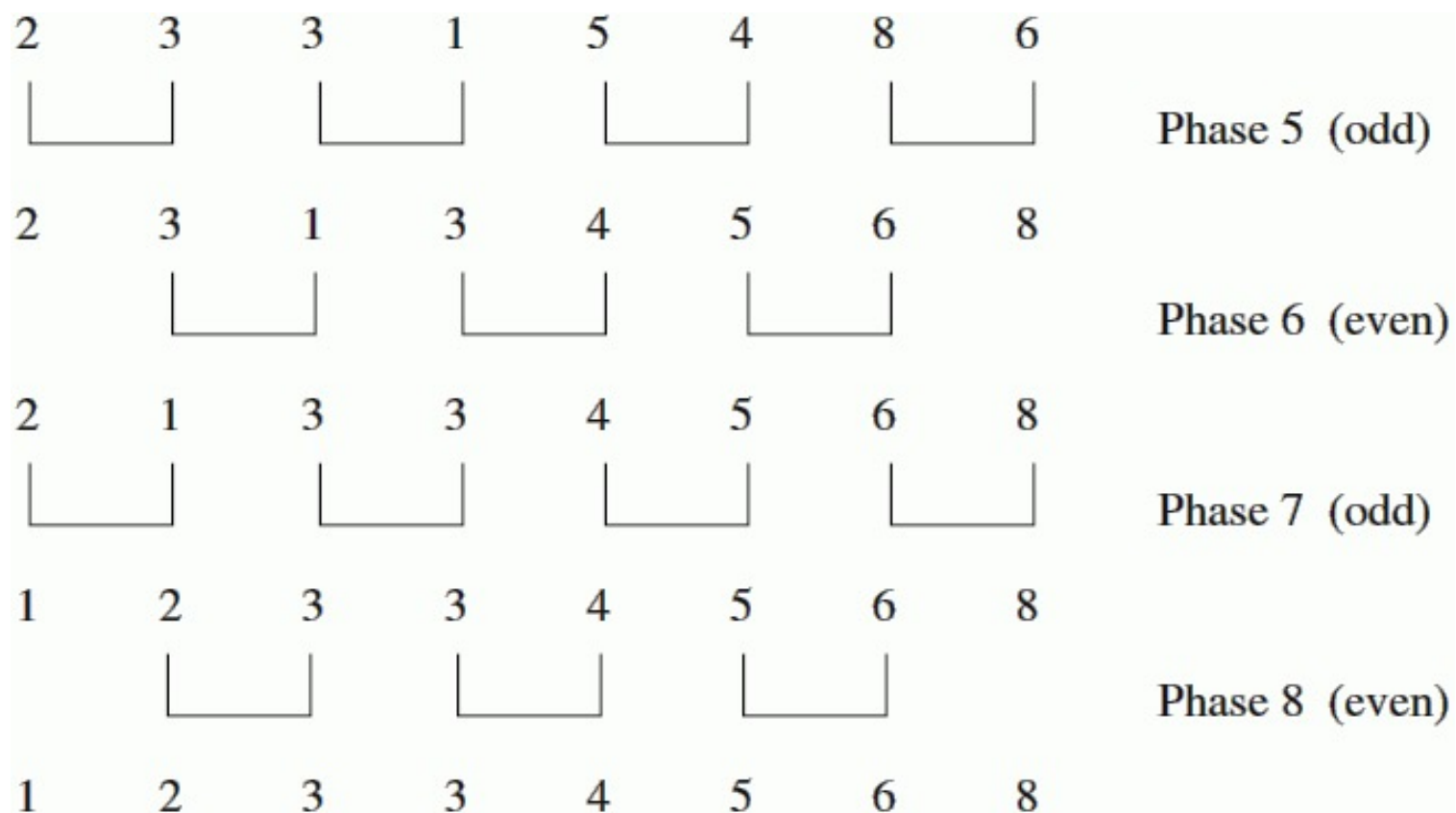
3.8 Odd-even转置排序

```
for (phase = 0; phase < n; phase++)  
    if (phase % 2 == 0)  
        for (i = 1; i < n; i += 2)  
            if (a[i-1] > a[i]) Swap(&a[i-1], &a[i]);  
    else  
        for (i = 1; i < n-1; i += 2)  
            if (a[i] > a[i+1]) Swap(&a[i],  
&a[i+1]);
```

3.8 Odd-even转置排序实例



3.8 Odd-even转置排序实例



3.8 Odd-even转置排序：OpenMP版本

```
for (phase = 0; phase < n; phase++) {  
    if (phase % 2 == 0)  
        #pragma omp parallel for num_threads(thread_count) \  
            default(none) shared(a, n) private(i, tmp)  
        for (i = 1; i < n; i += 2) {  
            if (a[i-1] > a[i]) {  
                tmp = a[i-1];  
                a[i-1] = a[i];  
                a[i] = tmp;  
            }  
        }  
}
```

3.8 Odd-even转置排序：OpenMP版本(2)

```

else
# pragma omp parallel for num_threads(thread_count) \
    default(none) shared(a, n) private(i, tmp)
    for (i = 1; i < n - 1; i += 2) {
        if (a[i] > a[i+1]) {
            tmp = a[i+1];
            a[i+1] = a[i];
            a[i] = tmp;
        }
    }
}

```

Q: 这个程序存在什么问题？

A: 频繁创建、销毁线程。

3.8 Odd-even转置排序：避免线程创建、销毁开销

```
# pragma omp parallel num_threads(thread_count) \
    default(none) shared(a, n) private(i, tmp, phase)
for (phase = 0; phase < n; phase++) {
    if (phase % 2 == 0)
#        pragma omp for
        for (i = 1; i < n; i += 2) {
...
        }
    else
#        pragma omp for
        for (i = 1; i < n - 1; i += 2) {
...
        }
}
```